System Analysis & Design

1. Problem Statement & Objectives

Problem

Many learners practise English with generic dialogues that ignore their interests, level and the words they already know. They find it hard to use vocabulary in realistic situations, to keep track of what they have learned, and many tools need a constant internet connection.

Solution

AI-Lingo generates personalised conversational scenarios and level-appropriate dialogues with AI, reuses previously learned vocabulary, adds only a few new expressions, and keeps saved words, progress and lesson snippets available offline.

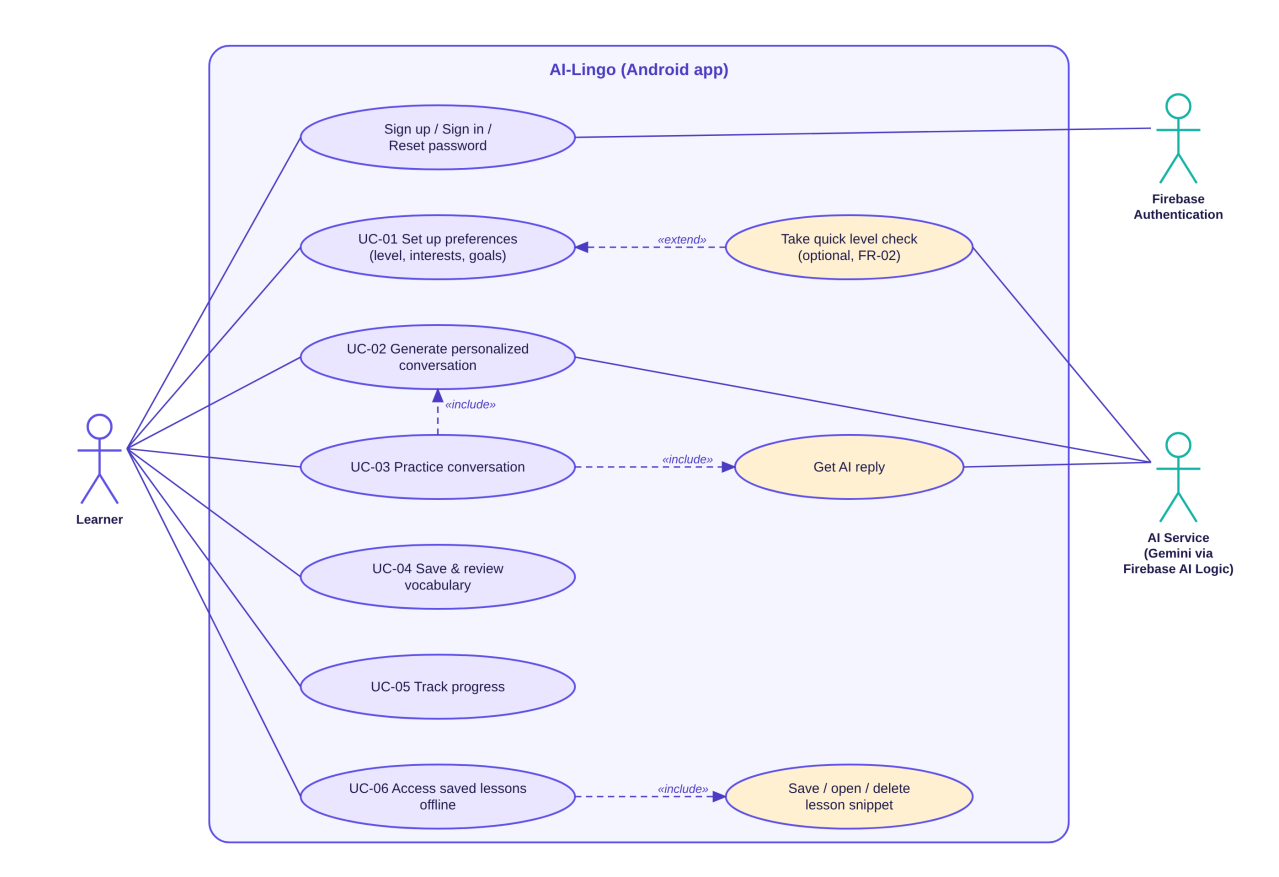
Objectives (6)

1. Personalise language learning by interests, goals and needs.
2. Adapt dialogue complexity to the learner's CEFR level (A1 to C2).
3. Apply Prompt Engineering (Prompt Chaining and Constrained Prompting).
4. Support real-world practice: everyday, academic and professional situations.
5. Track learning progress: vocabulary, progress and lesson snippets.
6. Ensure maintainability and data protection: modular object-oriented design, Cubit/BLoC state management, local Hive storage with AES encryption and the Android Keystore.

Scope of the first version: English only, text-based chat.

Out of scope: voice conversations, pronunciation scoring, advanced adaptive learning, fully offline AI generation, other languages.

1.1 Use Case Diagram & Descriptions



Use case diagram

Actors

Learner (primary actor); Firebase Authentication (external system); AI Service, Gemini via Firebase AI Logic (external system).

Use case descriptions

ID	Use case	Actor	Description
Auth	Sign up / Sign in / Reset password	Learner, Firebase Authentication	The learner creates an account or signs in with email and password, or resets a forgotten password.
UC-01	Set up preferences	Learner	The learner chooses level (optionally takes a quick level check), interests, goals, daily practice and chat-help style. The profile is saved on the device.
UC-02	Generate personalized conversation	Learner, AI Service	The learner picks a scenario. The app builds prompts from the profile and known words, the AI returns a dialogue, and the app checks and stores it. If there is no internet or the AI fails, a clear message and a retry button are shown.
UC-03	Practice conversation	Learner, AI Service	The learner replies in a text chat, the AI continues the dialogue, new expressions are highlighted, and the learner finishes the chat to get feedback.
UC-04	Save & review vocabulary	Learner	The learner saves new expressions, reviews them and marks them as learned. A word already saved is not duplicated.
UC-05	Track progress	Learner	The learner sees sessions, saved and learned words, and an empty state when there is no activity yet.
UC-06	Access saved lessons offline	Learner	Saved words, progress and lesson snippets open without an internet connection.

1.2 Functional & Non-Functional Requirements

Must = required in the first version; Could = optional; Proposed = suggested, pending team approval.

Functional requirements

FR-01	The learner can set and edit interests, goals and level, and they are saved	Must
FR-02	Optional quick level check that suggests a level	Could
FR-03	Generate scenarios (everyday, academic, professional) from interests and goals	Must
FR-04	Generate dialogues at the learner's level	Must
FR-05	Reuse learned vocabulary and add only a few new expressions	Must
FR-06	Check the AI response format before showing it	Proposed
FR-07	Content-safety check of generated text	Proposed
FR-08	Text-based chat with the AI dialogue	Must
FR-09	Short corrections of learner replies	Proposed
FR-10	Finish a session and record it in progress	Must
FR-11	Save vocabulary	Must
FR-12	Review vocabulary and mark as learned	Must
FR-13	Progress screen: sessions, words saved, words learned	Must
FR-14	Save, open and delete lesson snippets	Must
FR-15	Saved words, progress and snippets work offline	Must
FR-16	Clear message and retry when generation is unavailable	Must
FR-17	Use Prompt Chaining and Constrained Prompting	Must

Non-functional requirements

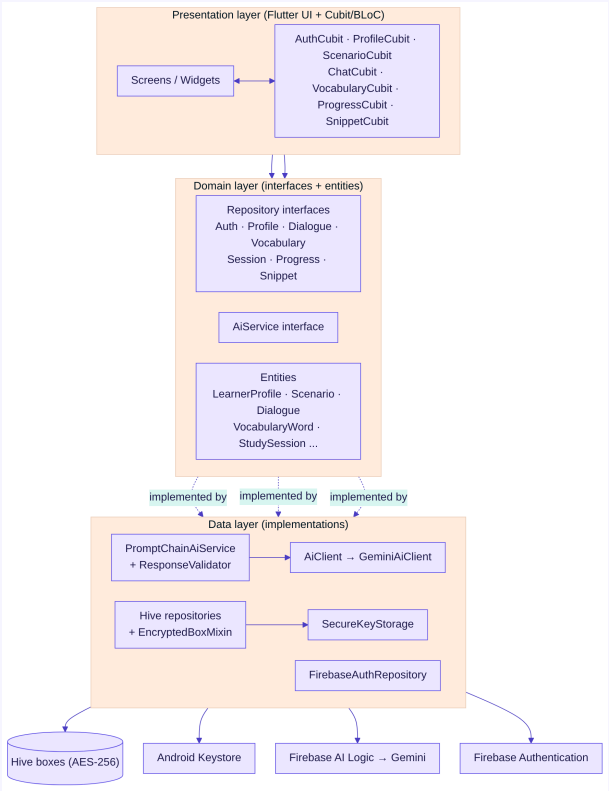
NFR-01	Modular object-oriented design (interfaces, mixins)	Must
NFR-02	State management with Cubit/BLoC	Must
NFR-03	Local data encrypted (AES) with the key protected by the Android Keystore	Must
NFR-04	Privacy: send only the data needed for generation to the AI	Proposed
NFR-05	Performance: loading indicator while content is generated	Proposed
NFR-06	No loss of saved data after restart or update	Must
NFR-07	Usability: a new learner finishes setup and one chat without help	Proposed
NFR-08	Minimum supported Android version	Proposed

Note: accounts use Firebase Authentication (added from the UI design).

1.3 Software Architecture

Architecture style: Layered architecture (Presentation, Domain, Data) with the Repository pattern and Cubit/BLoC for state management.

Layer	What it contains	Rule
Presentation	Flutter screens and widgets, Cubits (AuthCubit, ProfileCubit, ScenarioCubit, ChatCubit, VocabularyCubit, ProgressCubit, SnippetCubit)	Screens only talk to Cubits.
Domain	Entities (LearnerProfile, Scenario, Dialogue, VocabularyWord, StudySession and others), repository interfaces, AiService interface	Pure Dart, no Firebase or Hive code.
Data	Hive repositories with EncryptedBoxMixin, PromptChainAiService with ResponseValidator, AiClient (GeminiAiClient), FirebaseAuthRepository, SecureKeyStorage	Implements the domain interfaces, so the AI provider or storage can be replaced without changing the UI (NFR-01, NFR-02).



Layered architecture diagram

Key decisions

AI = Gemini through Firebase AI Logic (no API key inside the app); accounts = Firebase Authentication; local storage = Hive encrypted with AES-256, key kept with flutter_secure_storage backed by the Android Keystore.

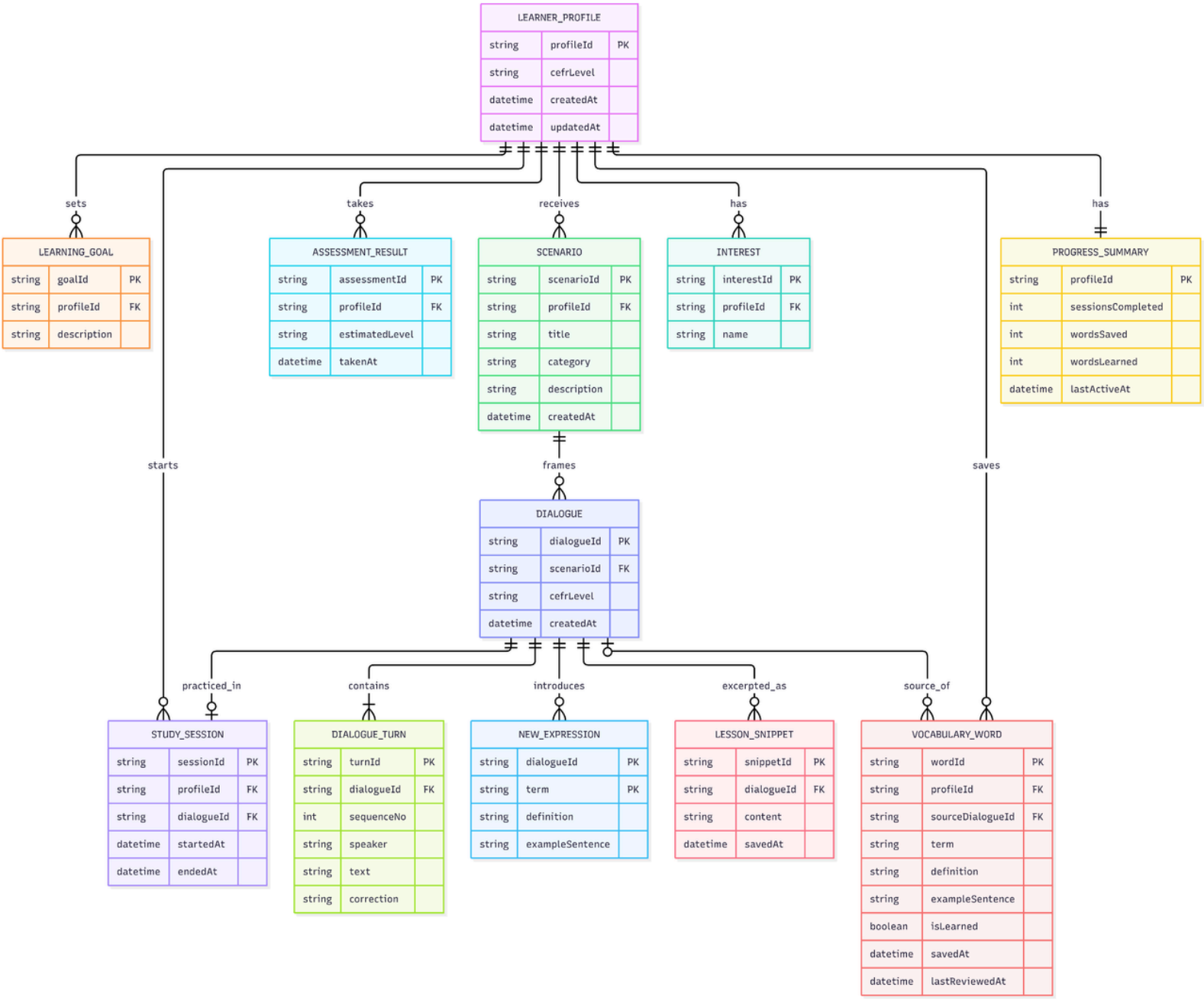
2. Database Design & Data Modeling

Draft based on the AI-Lingo SRS. Items marked (proposed) are pending team approval.

1. Design Assumptions

- One learner per device. User accounts are an open decision (SRS Section 7), so the profile is kept as a single record with a `profileId` that allows accounts to be added later.
- Hive is a NoSQL store with no joins, so the physical schema embeds child data inside its parent where they are always read together (interests inside the profile, turns inside a dialogue).
- AI generation needs a network connection. Once a dialogue is generated and stored, it can be opened offline.
- The proficiency assessment is optional (FR-02, Could), so `AssessmentResult` is optional too.
- The AI provider is undecided, so the AI is reached through an interface.

2.1 ER Diagram



Notes:

- `DIALOGUE_TURN.correction` is used only if corrections are approved (FR-09, proposed).
- `VOCABULARY_WORD.sourceDialogueId` is optional: a word may be saved from a dialogue or added by the learner.
- `LESSON_SNIPPET.content` is a copy of the saved text, so it stays available if the dialogue is later removed.

2.2 Logical Schema

The schema is in third normal form. Keys: PK = primary key, FK = foreign key.

Relation	Attributes	PK	FK
LearnerProfile	profileId, cefrLevel, createdAt, updatedAt	profileId	none
Interest	interestId, profileId, name	interestId	profileId references LearnerProfile
LearningGoal	goalId, profileId, description	goalId	profileId references LearnerProfile
AssessmentResult	assessmentId, profileId, estimatedLevel, takenAt	assessmentId	profileId references LearnerProfile
Scenario	scenarioId, profileId, title, category, description, createdAt	scenarioId	profileId references LearnerProfile
Dialogue	dialogueId, scenarioId, cefrLevel, createdAt	dialogueId	scenarioId references Scenario
DialogueTurn	turnId, dialogueId, sequenceNo, speaker, text, correction	turnId	dialogueId references Dialogue
NewExpression	dialogueId, term, definition, exampleSentence	dialogueId + term	dialogueId references Dialogue
VocabularyWord	wordId, profileId, sourceDialogueId, term, definition, exampleSentence, isLearned, savedAt, lastReviewedAt	wordId	profileId references LearnerProfile; sourceDialogueId references Dialogue (optional)
LessonSnippet	snippetId, dialogueId, content, savedAt	snippetId	dialogueId references Dialogue
StudySession	sessionId, profileId, dialogueId, startedAt, endedAt	sessionId	profileId references LearnerProfile; dialogueId references Dialogue
ProgressSummary	profileId, sessionsCompleted,	profileId	profileId references

	wordsSaved, wordsLearned, lastActiveAt		LearnerProfile

Rules:

- `category` is one of everyday, academic, professional. `speaker` is learner or ai. Levels use CEFR values A1 to C2.
- `term` should be unique per learner in VocabularyWord, so the same word is not saved twice.
- `ProgressSummary` can be recalculated from StudySession and VocabularyWord. It is stored so the progress screen opens quickly and works offline (FR-13, FR-15).

2.3 Physical Schema (Hive)

Each Hive box stores one kind of object through a generated TypeAdapter. Child records from the logical schema are embedded in their parent.

2.3.1 Boxes

Box	Key	Value type (typeId)	Embedded data	Offline
profileBox	`profile`	LearnerProfile (0)	interests, goals, assessment results	Yes
vocabularyBox	wordId	VocabularyWord (1)	none	Yes
snippetsBox	snippetId	LessonSnippet (2)	none	Yes
sessionsBox	sessionId	StudySession (3)	none	Yes
progressBox	`progress`	ProgressSummary (4)	none	Yes
dialoguesBox	dialogueId	Dialogue (5)	scenario (8), turns (6), new expressions (7)	Yes, once stored

2.3.2 Types and fields

Each field is listed as HiveField index, name, and Dart type. The type IDs and indexes are proposed and must never be reused once released.

Type (typeId)	Fields
LearnerProfile (0)	0 profileId: String; 1 cefrLevel: String; 2 interests: List of String; 3 goals: List of String; 4 assessmentResults: List of AssessmentResult; 5 createdAt: DateTime; 6 updatedAt: DateTime
VocabularyWord (1)	0 wordId: String; 1 term: String; 2 definition: String; 3 exampleSentence: String; 4 isLearned: bool; 5 sourceDialogueId: String (nullable); 6 savedAt: DateTime; 7 lastReviewedAt: DateTime (nullable)
LessonSnippet (2)	0 snippetId: String; 1 dialogueId: String; 2 content: String; 3 savedAt: DateTime
StudySession (3)	0 sessionId: String; 1 dialogueId: String; 2 startedAt: DateTime; 3 endedAt: DateTime (nullable until the session ends)
ProgressSummary (4)	0 sessionsCompleted: int; 1 wordsSaved: int; 2 wordsLearned: int; 3 lastActiveAt: DateTime
Dialogue (5)	0 dialogueId: String; 1 scenario: Scenario; 2 cefrLevel: String; 3 turns: List of DialogueTurn; 4 newExpressions: List of NewExpression; 5 createdAt: DateTime
DialogueTurn (6)	0 sequenceNo: int; 1 speaker: String; 2 text: String; 3 correction: String (nullable)
NewExpression (7)	0 term: String; 1 definition: String; 2 exampleSentence: String
Scenario (8)	0 scenarioId: String; 1 title: String; 2 category: String; 3 description: String; 4 createdAt: DateTime
AssessmentResult (9)	0 estimatedLevel: String; 1 takenAt: DateTime

2.3.3 Storage rules

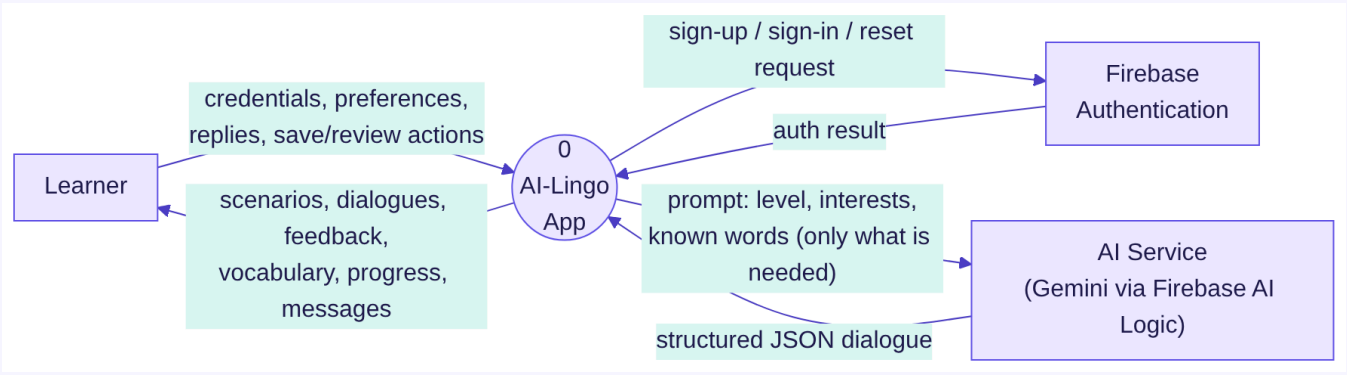
- **Encryption (NFR-03):** every box is opened with an AES cipher. The key is created on first launch and stored in the Android Keystore. The exact method and plugin are to be reviewed with the team.
- **Queries:** Hive has no secondary indexes, so lists such as words not yet learned are filtered in memory. The data volume per learner is small.

- **Atomic updates:** when a session ends, the session and the progress summary are written together so the counts never disagree.
- **Versioning (proposed):** store a schema version in the profile box so a later app update can migrate the data.

3. Data Flow Diagrams & Data Modeling

3.1 Data Flow Diagrams

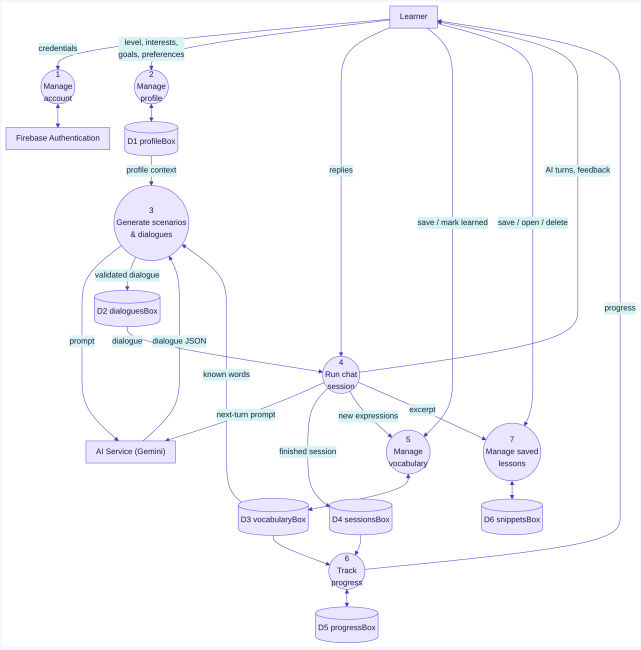
DFD level 0 (context)



DFD level 0 context diagram

The app exchanges data with three external entities: the Learner, Firebase Authentication and the AI Service.

DFD level 1



DFD level 1 diagram

Processes

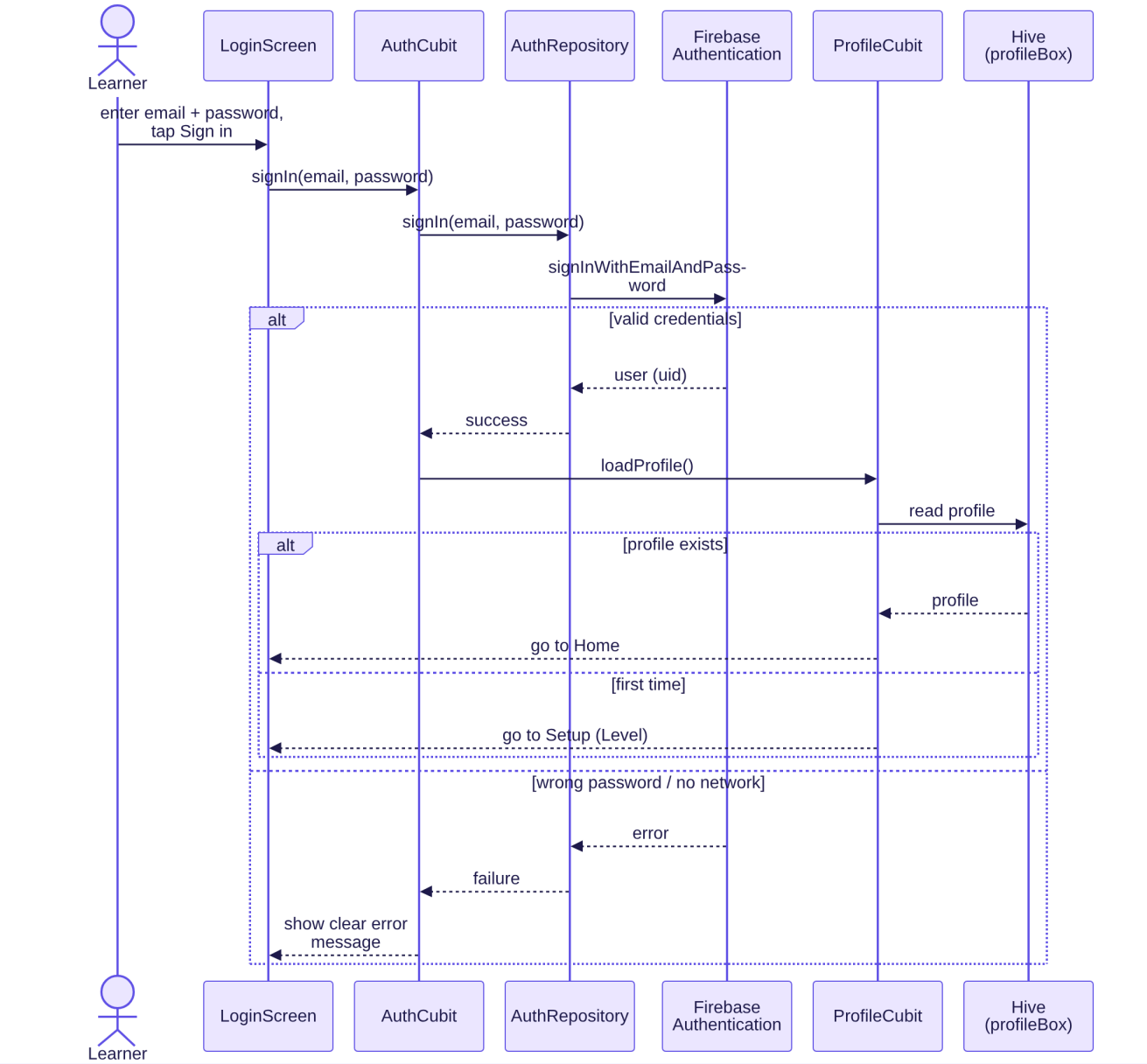
1 Manage account, 2 Manage profile, 3 Generate scenarios and dialogues, 4 Run chat session, 5 Manage vocabulary, 6 Track progress, 7 Manage saved lessons.

Data stores (Hive boxes)

D1 profileBox, D2 dialoguesBox, D3 vocabularyBox, D4 sessionsBox, D5 progressBox, D6 snippetsBox.

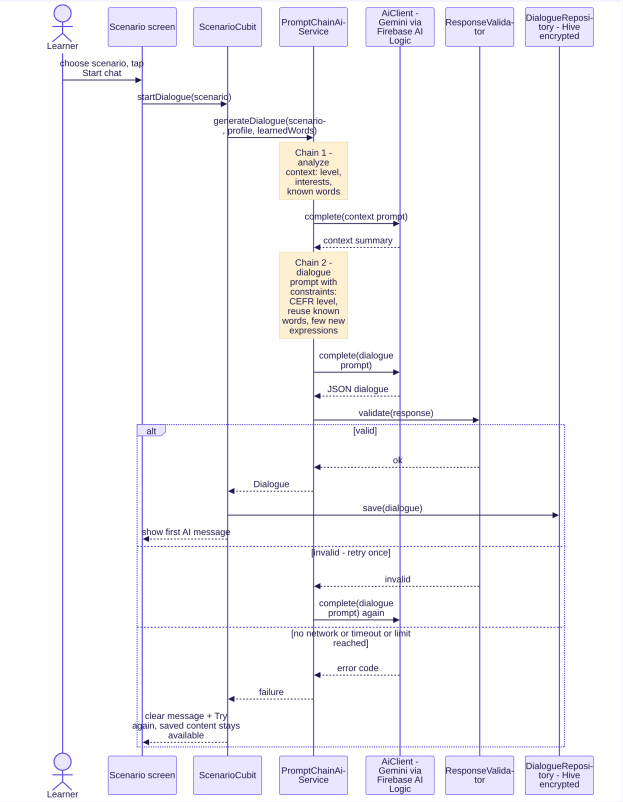
3.2 Sequence Diagrams

1 Sign in



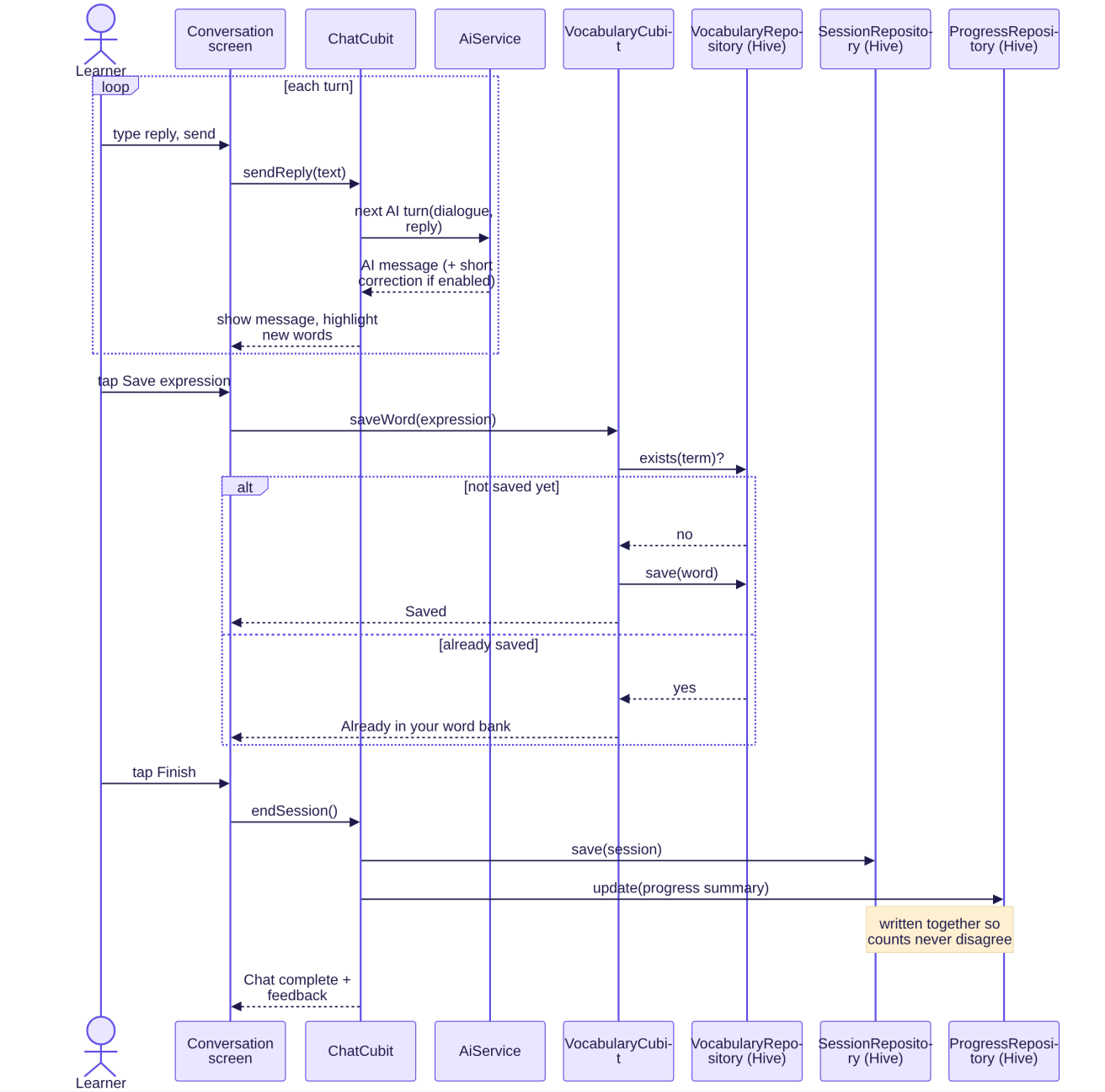
Sign-in sequence diagram

2. Generate a personalised dialogue (Prompt Chaining and Constrained Prompting)



Personalised dialogue generation sequence diagram

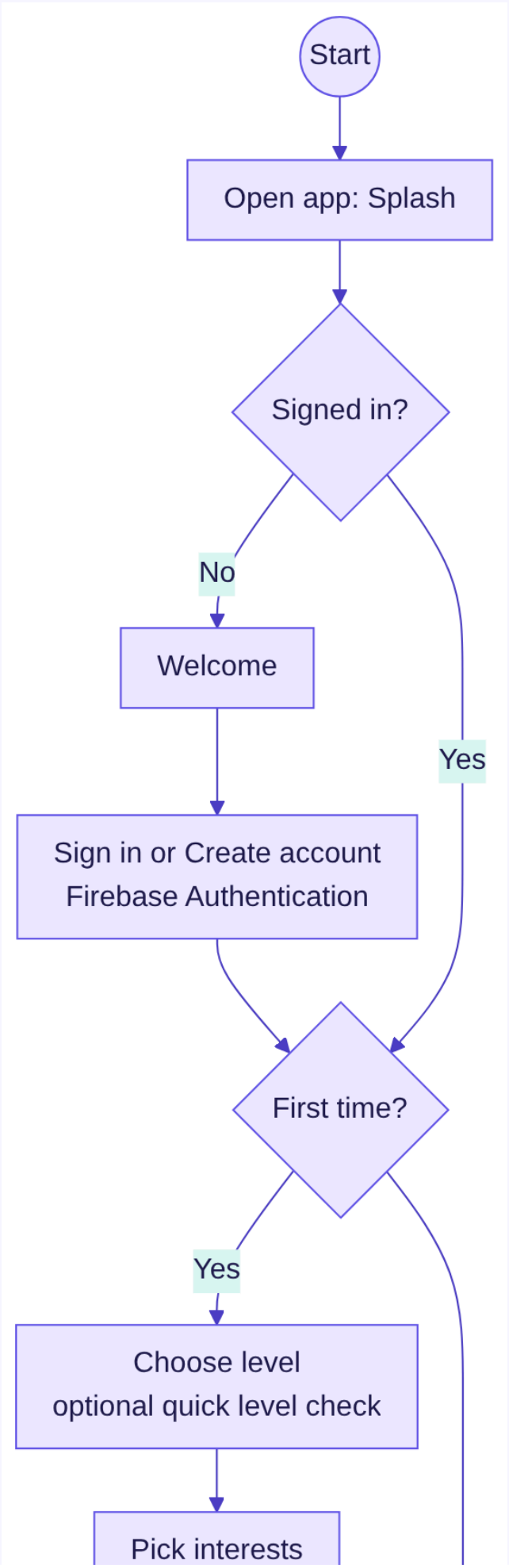
3. Practice a conversation and save a word

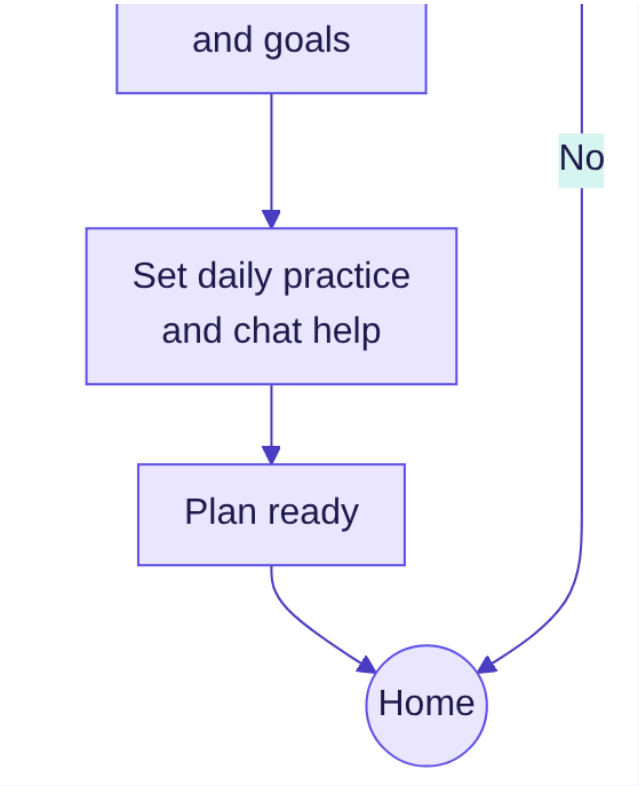


Chat and vocabulary save sequence diagram

3.3 Activity Diagrams

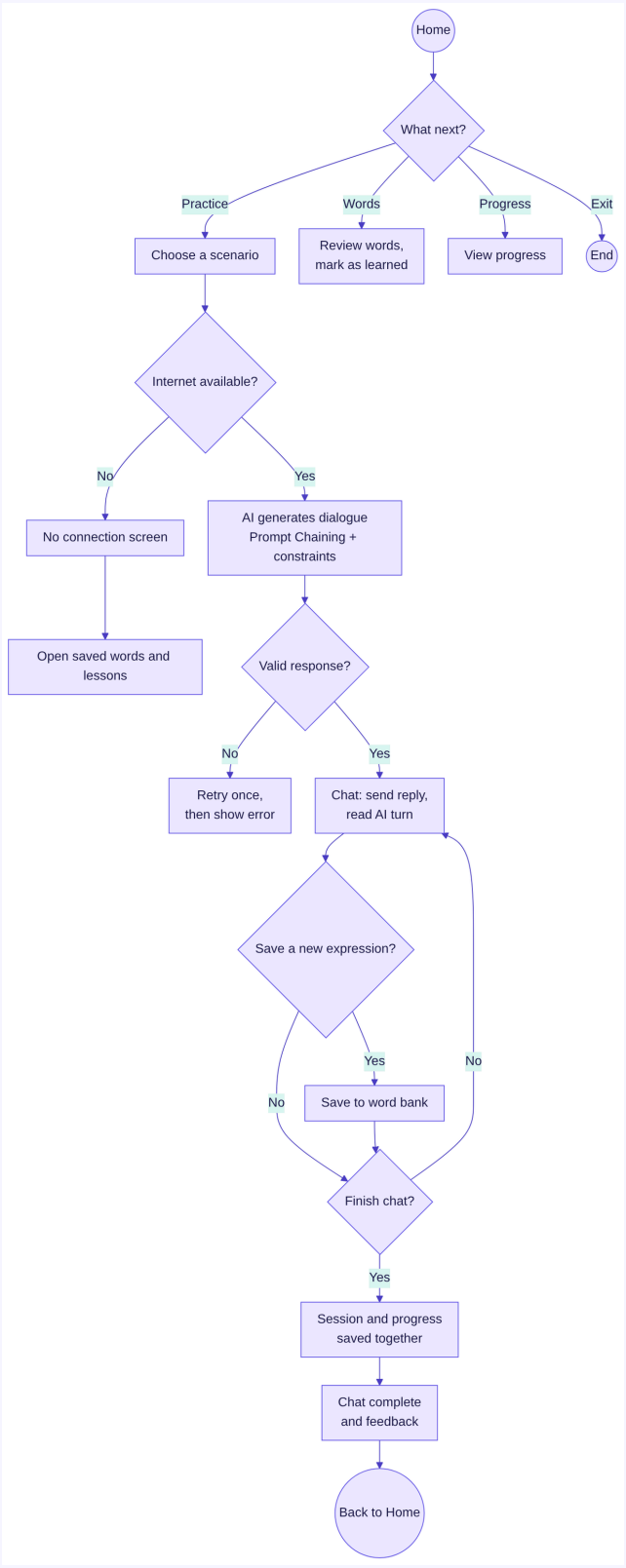
Onboarding and sign-in


Start
Open app: Splash
Signed in?
No
Welcome
Yes
Sign in or Create account
Firebase Authentication
First time?
Yes
Choose level
optional quick level check
Pick interests



Onboarding and sign-in activity diagram

Practice, words and progress

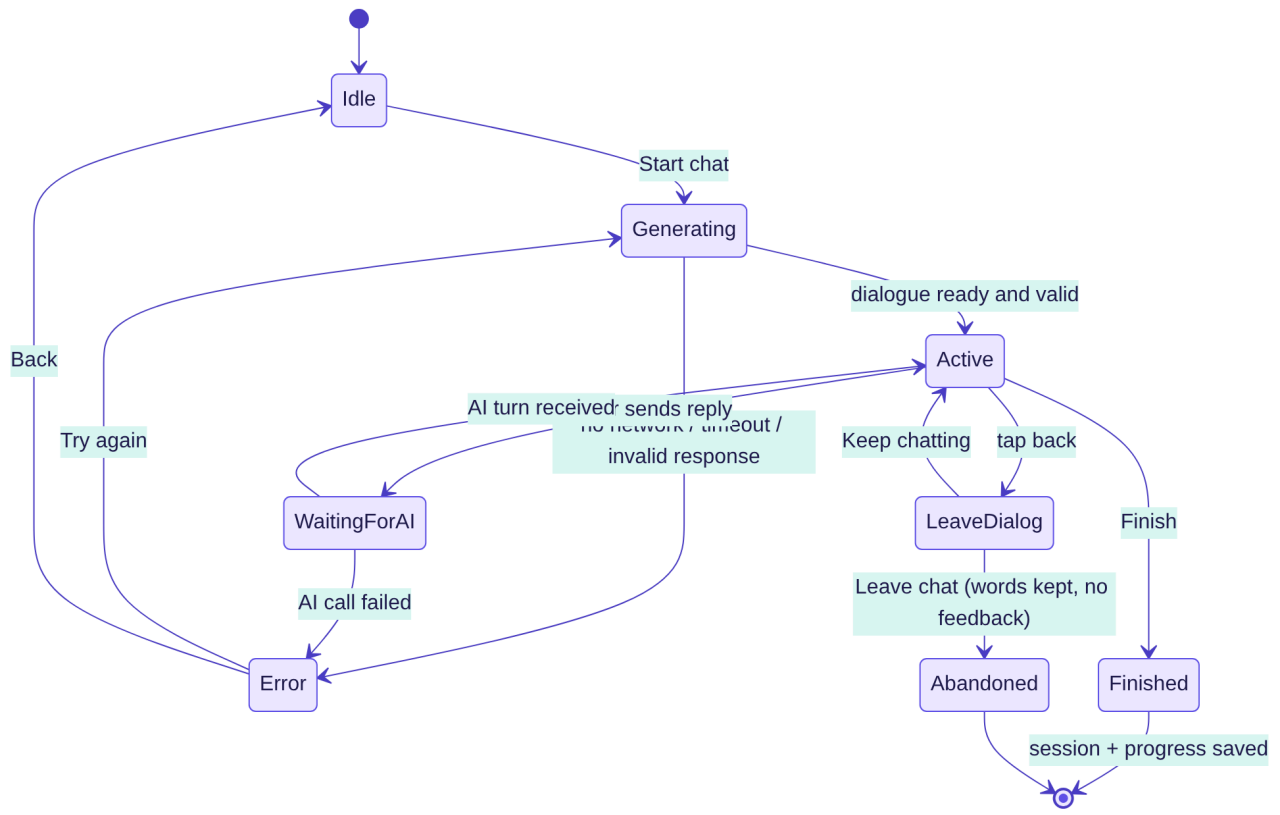


Practice, words and progress activity diagram

3.4 State Diagrams

Chat session states

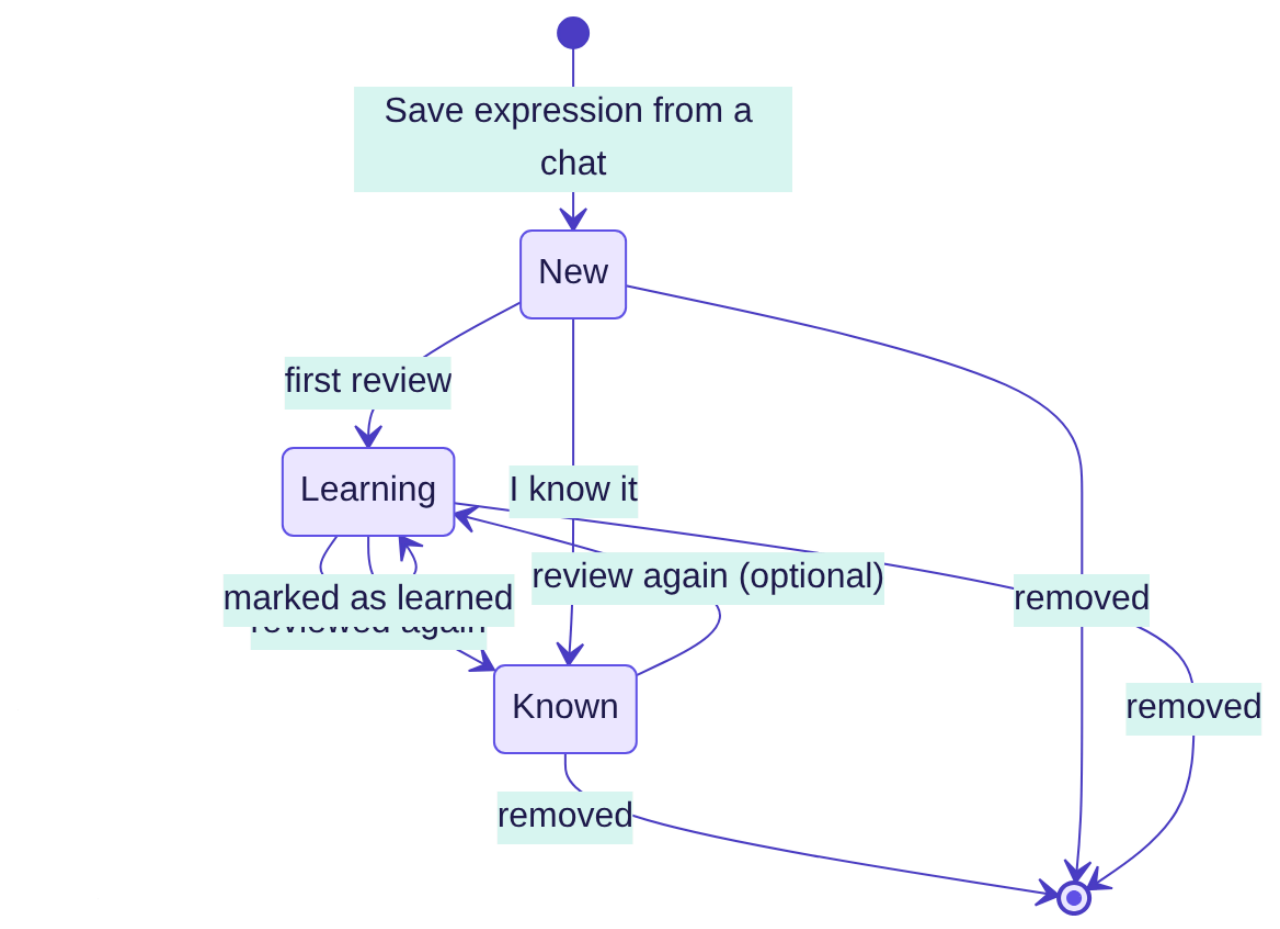
Idle, Generating, Active, WaitingForAI, Error, LeaveDialog, Abandoned, Finished.



Chat session state diagram

Vocabulary word states

New, Learning, Known



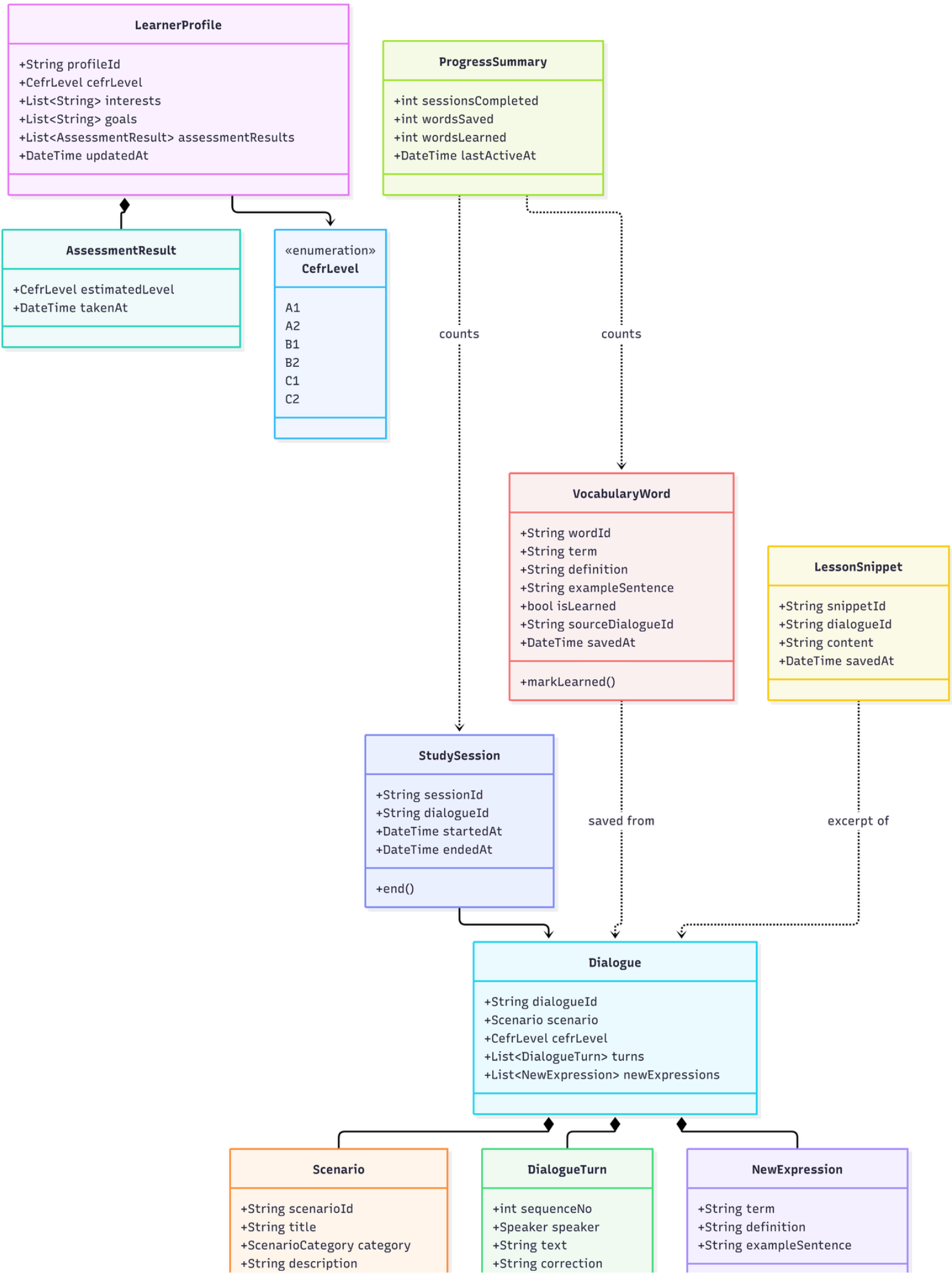
Vocabulary word state diagram

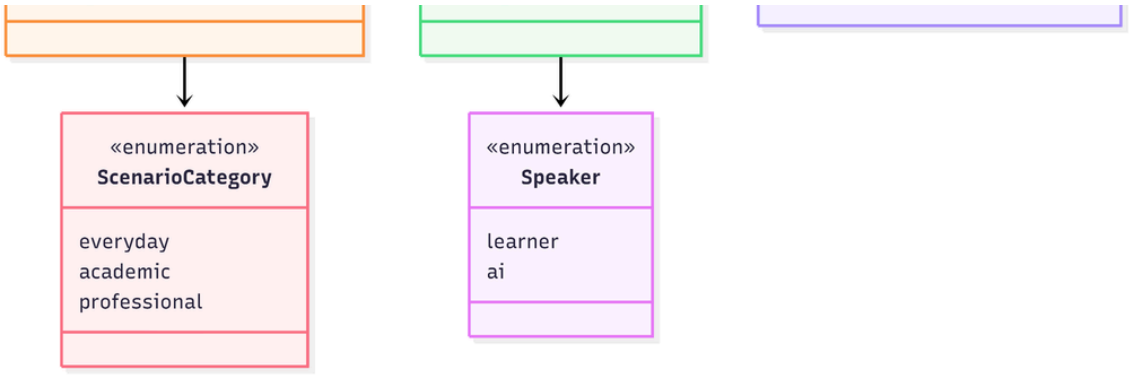
The exact rule for marking a word as learned is still to be confirmed by the team.

3.5 Class Diagrams

3.5.1 Domain model

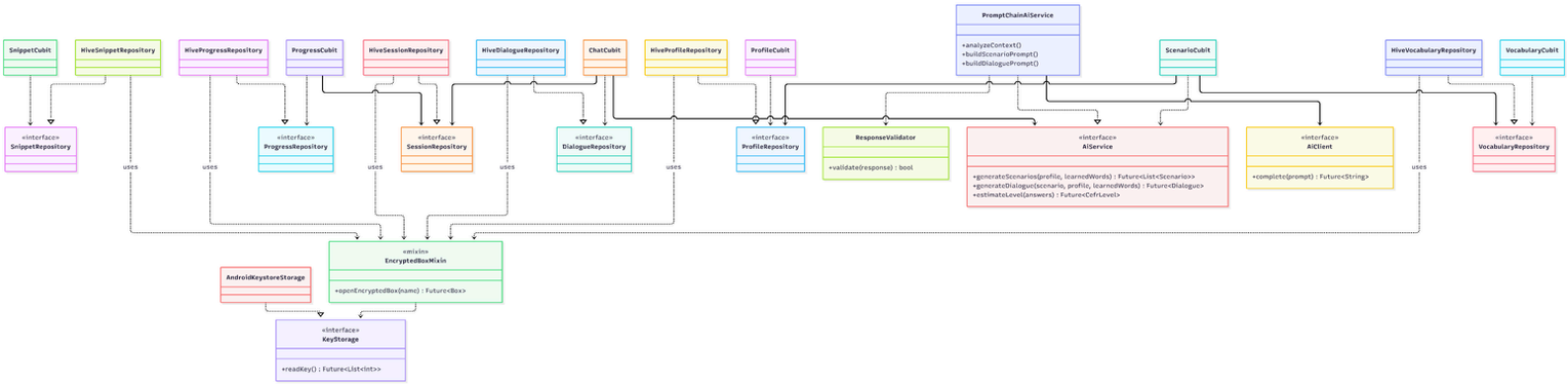
These classes match the Hive types in Section 4.


LearnerProfile
+String profileId
+CefrLevel cefrLevel
+List<String> interests
+List<String> goals
+List<AssessmentResult> assessmentResults
+DateTime updatedAt
ProgressSummary
+int sessionsCompleted
+int wordsSaved
+int wordsLearned
+DateTime lastActiveAt
AssessmentResult
+CefrLevel estimatedLevel
+DateTime takenAt
«enumeration»
CefrLevel
A1
A2
B1
B2
C1
C2
counts
counts
VocabularyWord
+String wordId
+String term
+String definition
+String exampleSentence
+bool isLearned
+String sourceDialogueId
+DateTime savedAt
+markLearned()
LessonSnippet
+String snippetId
+String dialogueId
+String content
+DateTime savedAt
StudySession
+String sessionId
+String dialogueId
+DateTime startedAt
+DateTime endedAt
+end()
saved from
excerpt of
Dialogue
+String dialogueId
+Scenario scenario
+CefrLevel cefrLevel
+List<DialogueTurn> turns
+List<NewExpression> newExpressions
Scenario
+String scenarioId
+String title
+ScenarioCategory category
+String description
DialogueTurn
+int sequenceNo
+Speaker speaker
+String text
+String correction
NewExpression
+String term
+String definition
+String exampleSentence



3.5.2 Application architecture

The app has three layers. Cubits (presentation) depend only on interfaces (domain). Hive and the AI service (data) implement those interfaces, so the AI provider and the storage can be replaced without changing the UI (NFR-01, NFR-02).



Notes:

- `AiClient` hides the AI provider, which is still undecided. `PromptChainAiService` applies Prompt Chaining and Constrained Prompting (FR-17); how is to be reviewed with the team.
- `ResponseValidator` is only needed if the format check is approved (FR-06, proposed).
- `estimateLevel` is only needed if the assessment is included (FR-02).
- `EncryptedBoxMixin` is shared by all Hive repositories so encryption is written once (NFR-03).

4. UI/UX Design & Prototyping

4.1 Wireframes & Mockups

The app has 28 mockup screens in Figma, grouped into flows:

Entry & Auth (Splash, Welcome, Sign in, Create account, Forgot password, Check your email)	Setup (Level, Level check, Level result, Interests, Goals, Preferences, Plan ready)	Learning (Home, Practice, Scenario intro, Conversation, Leave chat, Chat complete, Feedback)	Words and Progress (Vocabulary, Review words, Word detail, Saved offline, Progress, Profile, Edit profile, No connection)

Wireframes and the full UI/UX design are available in the project's Figma file: [Figma]

4.2 UI/UX Guidelines

Principles:

1 Simple – one main action per screen	**2 Consistent** – same colors, fonts and buttons everywhere	**3 Friendly** – short texts and clear messages when something goes wrong	**4 Accessible** – readable text and large tap targets

Colors table:

Color	Use	Hex
Primary	(buttons, selected items)	#6254E8
Primary dark		#4B3FC4
Text		#1E1B4B
Background		#F6F5FF
Known word		#14B8A6
New word		#FF9F5A

Typography: Sora for titles (Bold 26-28), Plus Jakarta Sans for body text (Medium 13-15) and buttons (ExtraBold 12-17).

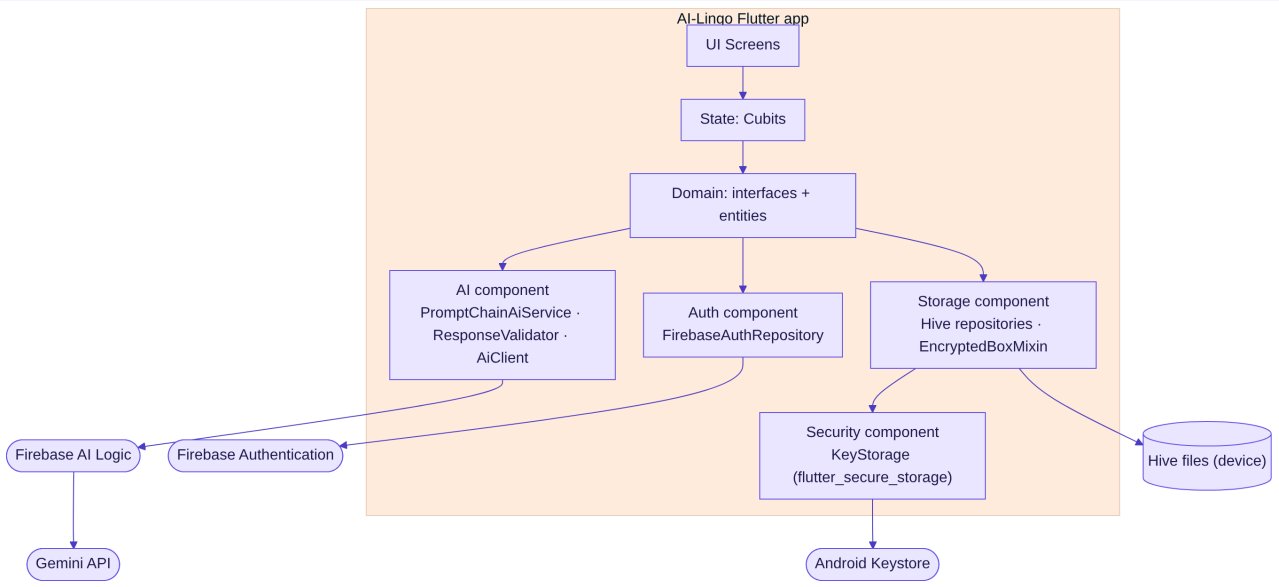
Accessibility: text contrast at least 4.5:1 (white on #6254E8 = 5.31:1, passes WCAG AA), tap targets at least 44 px, clear error messages.

5. System Deployment & Integration: Technology Stack

Area	Technology	Purpose
Frontend / app	Flutter and Dart	Cross-platform UI, Android first.
State management	Cubit / BLoC (flutter_bloc)	Screen state and learning progress.
Local database	Hive (NoSQL)	Vocabulary, progress, sessions, dialogues, snippets, profile; works offline.
Security	AES-256 encryption, flutter_secure_storage, Android Keystore	Protect local data and the encryption key.
Authentication	Firebase Authentication (email and password)	Sign up, sign in, reset password.
AI	Gemini through Firebase AI Logic	Generates scenarios and dialogues; no API key in the app.
Prompting	Prompt Chaining and Constrained Prompting	Level-appropriate, vocabulary-aware dialogues.
Design	Figma and Canva	UI design and documentation.
Testing	flutter_test, bloc_test, integration_test	Unit, widget and integration tests.
Version control	GitHub (GitHub Actions optional)	Source code and documentation.

There is no custom backend server.

Component Diagram



Component diagram

Short description

The UI uses Cubits, Cubits use domain interfaces, and the AI, authentication and storage components implement them. The storage component uses the security component to get the encryption key from the Android Keystore.

6.1 API Documentation

The app has no backend of its own. It uses three services.

Service	Purpose	Details
Firebase Authentication	sign up, sign in, reset password	email and password
Gemini via Firebase AI Logic	generates scenarios and dialogues from the learner's level, interests and known words	no API key inside the app; free tier on the Spark plan (limits vary, check Google's docs)
Hive (local database)	saves profile, vocabulary, progress and lesson snippets	encrypted with AES-256; the key is kept in the Android Keystore through flutter_secure_storage.

The app talks to the AI through one interface (AiService), so the provider can be changed later by editing one class.

Example request:

```
{"level":"A2","scenario":"cafe-order","knownWords":["recommend"],"maxNewWords":3}
```

Example response:

```
{"turns":[{"speaker":"ai","text":"Hi! What can I get for you?"}],"newExpressions":
[{"word":"latte","definition":"A coffee made with hot milk."}]}
```

Errors handled:

no internet, timeout, service error, usage limit reached, invalid response.

In every case saved content stays available offline.

6.2 Testing & Validation

We test at three levels: unit tests (Cubits, repositories), widget tests (screens) and one integration test for the main flow. At the end, 5 learners try the app (user acceptance test).

ID	Test	Expected result
TC-01	Create a profile and restart the app	Profile is restored.
TC-02	Generate a dialogue at level A2	Difficulty matches the level.
TC-03	Reply in the chat	Next AI message appears.
TC-04	Save a word and restart	Word is still in the list.
TC-05	Open saved content in airplane mode	Works normally.
TC-06	Generate a dialogue with no internet	Clear message, no data lost.
TC-07	Open the Hive files outside the app	Data is not readable.
TC-08	Open Progress with no activity	Empty state with a start button.

6.3 Deployment Strategy

Platform:	Android app built with Flutter.
Release:	flutter build apk --release, signed with our own keystore, tested on two devices.
Hosting:	Firebase (Authentication and AI Logic); no server to maintain.
Scaling:	the free-tier limits are the main limit; if reached, the app shows a message and lets the learner use saved lessons.
Optional:	a GitHub Actions workflow that runs flutter analyze and flutter test on every pull request.